Internship Report On

“NTPWEdit Clone - CLI-Based Windows Password Management Tool”

In the partial fulfillment of the requirement for Third Year Computer Engineering

Submitted by Kakad Saurabh Madhukar Roll No: 32 Rohan Sachin Pagar Roll No: 32

Under the Guidance of Prof. P. P. Kakade

DEPARTMENT OF COMPUTER ENGINEERING

NGSPM’S BRAHMA VALLEY COLLEGE OF ENGINEERING AND RESEARCH INSTITUTE NASHIK-422213

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE 2025-2026

ACKNOWLEDGEMENT

It is a great pleasure for us to express our deep sense of gratitude to all those who have helped us directly and indirectly in the successful completion of this internship project. Technical projects of this nature require not only theoretical knowledge but also a significant amount of hands-on experimentation and research into undocumented systems.

We would like to express our special thanks of gratitude to our guide, **Prof. P. P. Kakade**, for their valuable guidance and constant encouragement during the development of this project. Their insights into systems programming and technical expertise were instrumental in overcoming the various challenges we faced, especially regarding the binary parsing of system files and the transition to a robust command-line architecture.

We also express our sincere thanks to the **H.O.D. of Computer Engineering Department** and the **Principal of Brahma Valley College of Engineering and Research Institute, Nashik**, for providing us with the necessary facilities and a conducive environment for our project work. The resources provided by the institution allowed us to dive deep into the project requirements.

Finally, we would like to thank our parents and friends for their continuous support and motivation throughout the internship duration. Their encouragement was a great source of strength during the debugging phases of this complex utility.

Kakad Saurabh Madhukar Rohan Sachin Pagar

ABSTRACT

The Security Accounts Manager (SAM) hive is a critical component of the Windows NT operating system family, storing sensitive user account information, including password hashes and group memberships. Management of these accounts typically requires the operating system to be active; however, in scenarios involving system lockouts, forgotten administrative credentials, or forensic investigations, offline access is required.

This internship project, titled **"NTPWEdit Clone (CLI Version)"**, involves the development of a lightweight, standalone C++ command-line utility designed to parse and manipulate Windows Registry hives directly at the binary level. By bypassing the Windows Registry APIs (which are restricted by kernel-level locks on active systems), the tool can operate on SAM files located on non-booted drives. This is achieved through a custom-built registry parsing engine that treats the hive as a raw binary database.

The project demonstrates a modular approach to system-level programming, featuring a custom registry hive parsing engine (`Hive` class), a SAM-specific data manager (`SAMManager`), and a robust Command Line Interface (`main_cli.cpp`). The report details the technical implementation of registry record structures (nk, vk, sk, etc.), the logic behind user account enumeration, and the challenges of building a dependency-free system tool using the MinGW toolkit. The resulting utility automates SAM detection and provides a high-performance console interface for account visualization, serving as a powerful tool for system administrators and security professionals.

INDEX

Sr. No.	Content	Page No.
1	**INTRODUCTION**	**1**
1.1	Project Overview and Context	1
1.2	Detailed Problem Statement	3
1.3	Objectives of the Project	5
1.4	Scope and Technical Boundaries	7
1.5	Development Methodology	9
2	**LITERATURE SURVEY / BACKGROUND**	**12**
2.1	Evolution of Configuration Systems in Windows	12
2.2	Windows Security Architecture: An In-Depth View	15
2.3	The Role of the LSASS and SAM Modules	18
2.4	Cryptographic Evolution: From LM to NTLMv2	21
2.5	Theoretical History of Registry Research	24
3	**REGISTRY HIVE FORMAT DEEP DIVE**	**28**
3.1	The Binary Philosophy of Registry Bins	28
3.2	Detailed Analysis of the REGF Header	31
3.3	Cell Management and Memory Segmentation	34
3.4	Structural Logic of Named Key (nk) Records	37
3.5	Value Key (vk) and Data Indirection	40
3.6	Search Optimization: Hash Lists (lh/lf)	43
4	**SYSTEM FLOW AND ARCHITECTURE**	**47**
4.1	High-Level Operational Sequence (CLI)	47
4.2	Component-Based Architecture and Layering	50
4.3	Resiliency and Error Handling Philosophy	53

4.4	Functional Flow Diagrams	56
5	**TECHNICAL IMPLEMENTATION - REGISTRY ENGINE**	**60**
5.1	Registry Engine (ntreg.cpp) Logic Walkthrough	60
5.2	Cell Level Access and Binary Pointer Arithmetic	63
5.3	Enumerate Subkeys Algorithm Analysis	66
5.4	Value Key Retrieval and Data Type Mapping	69
6	**TECHNICAL IMPLEMENTATION - SAM MANAGER**	**73**
6.1	SAM Management (sam.cpp) Responsibilities	73
6.2	Recursive Path Resolution in Hierarchical Hives	76
6.3	User Account Enumeration and RID Mapping	79
6.4	Forensic Analysis of V and F records	82
7	**COMMAND LINE INTERFACE ARCHITECTURE**	**86**
7.1	CLI Design Philosophy and Portability Goals	86
7.2	Argument Parsing and Option Configuration	89
7.3	Standard Output Formatting and Table Logic	92
7.4	Automatic Path Detection Logic for System Config	95
8	**BUILD SYSTEM AND AUTOMATION**	**99**
8.1	MinGW Toolkit Implementation and Toolchain	99
8.2	Static Linking for Dependency-Free Portability	102
8.3	Compilation for Windows Console Subsystem	105
9	**TROUBLESHOOTING AND FAQ**	**109**
9.1	Common Error Codes and Resolutions	109
9.2	Permission and Lock Exceptions Management	112
10	**RESULTS AND PERFORMANCE**	**116**
10.1	Functional Verification and Test Case Summary	116
10.2	Performance Benchmarks and Load Analysis	119
11	**DISCUSSION AND SECURITY**	**123**
11.1	Ethical Considerations in Security Software	123
11.2	The Impact of Full Disk Encryption (BitLocker)	126
11.3	Technical Challenges and Future Resilience	129

12	**CONCLUSION**	**133**
13	**FUTURE SCOPE AND ENHANCEMENTS**	**136**
14	**APPENDICES (FULL SOURCE CODE)**	**140**
14.1	Appendix A: ntreg.h	140
14.2	Appendix B: ntreg.cpp	142
14.3	Appendix C: sam.h	146
14.4	Appendix D: sam.cpp	148
14.5	Appendix E: main_cli.cpp	150
14.6	Appendix F: crypto.h / crypto.cpp	154
14.7	Appendix G: build.bat	156
15	**GLOSSARY AND REFERENCES**	**160**

CHAPTER 1: INTRODUCTION

1.1 Project Overview and Context

The Windows Registry is a hierarchical database that stores configuration settings and options on Microsoft Windows operating systems. It is the core repository for hardware settings, operating system parameters, user preferences, and system-wide security policies. Historically, interacting with the registry required either the Windows API or the built-in `Regedit` utility. However, both of these methods are inherently tied to the active state of the operating system.

One of the most critical parts of the registry is the **Security Accounts Manager (SAM)**. The SAM hive stores local user accounts and encrypted password hashes. When Windows is running, the SAM hive is loaded and exclusively locked by the kernel. This prevents any other process from reading or modifying the file. This creates a significant challenge for system recovery or forensic auditing when the system cannot be booted.

The **"NTPWEdit Clone (CLI Version)"** project addresses this challenge by providing a standalone C++ command-line tool that parses the binary registry hives directly from the disk. By bypassing the Windows OS and its associated API restrictions, this tool can list user accounts and potentially modify account status on offline drives—such as those mounted in recovery environments or forensic workstations.

1.2 Detailed Problem Statement

System administrators and security professionals frequently encounter situations where standard Windows authentication is no longer accessible:

1. **Forgotten Administrative Passwords**: Without a secondary administrative account or a domain-level reset mechanism, a single forgotten password can result in the loss of access to an entire system.
2. **Disabled Accounts**: Security incidents or administrative errors can lead to a state where the local 'Administrator' account is disabled, leaving no way to gain elevated privileges through the standard login screen.
3. **Forensic Dead-Disk Analysis**: Digital investigators must often audit a computer's state without ever booting the operating system, to ensure no evidence is altered. They require a tool that can extract a list of

users and their security status directly from the raw hive file.
4. **Minimal Environment Support**: GUI-based tools often fail in lightweight recovery environments (like minimal WinPE or Linux shells). A CLI-based tool is essential for reliability and scriptability in these constrained environments.

1.3 Objectives of the Project

1. **Develop a Robust Binary Registry Engine**: Implement a C++ module capable of mapping little-endian binary structures (regf, hbin, nk, vk) into accessible data structures.
2. **Automate Local Security Discovery**: Logic to identify the system's SAM file path across different Windows architectures (x86/x64).
3. **Accurate Account Enumeration**: Successfully navigate the SAM hierarchical path to extract usernames and their associated Relative Identifiers (RIDs).
4. **CLI Formatted Output**: Present data in a clean, tabular console format for easy reading and potential automation via piping.
5. **Zero-Dependency Portability**: Utilize static linking to ensure the tool runs on any Windows NT distribution from Windows XP to Windows 11 without requiring external runtime libraries.

CHAPTER 7: COMMAND LINE INTERFACE ARCHITECTURE

7.1 CLI Design Philosophy and Portability Goals

Unlike GUI-based tools which require a full desktop subsystem, a CLI tool is designed for maximum portability. By targeting the Console Subsystem, our tool can run in:

- Standard Windows Command Prompts.
- PowerShell and Windows Terminal.
- Windows Recovery Environment (WinRE) Command Prompt.
- Minimal Windows PE (Preinstallation Environment) instances.

The goal is to provide a "no-frills," high-reliability interface that responds instantly and can be easily documented and automated.

7.2 Argument Parsing and Option Configuration

The CLI follows standard POSIX-like argument conventions.

- **-p, --path**: Allows the user to point the tool at an external SAM hive (e.g., from a drive D: mounted in recovery).
- **-h, --help**: Provides immediate documentation of available commands.

The implementation in `main_cli.cpp` uses a robust loop-based parser to handle these inputs safely, ensuring that missing arguments or invalid flags are reported with a helpful usage message.

7.3 Standard Output Formatting and Table Logic

To ensure readability, the tool uses the `<iomanip>` header to output account data in a structured table.

- **Left Alignment**: Usernames are aligned to the left with a fixed width of 30 characters.

- **Hexadecimal Formatting**: RIDs are printed in their native hex format (e.g., 0x1F4) for easier correlation with SID documentation and registry forensics.

7.4 Automatic Path Detection Logic for System Config

To reduce user friction, the tool uses the `GetSystemDirectory` API to identify where Windows is installed. It then appends `\\config\\SAM` to this path. This allows a user on a live (elevated) system to simply run the tool without any arguments to see their local account list immediately.

CHAPTER 14: APPENDICES (FULL SOURCE CODE)

14.5 Appendix E: main_cli.cpp

```
#include <iostream>
#include <vector>
#include <string>
#include <iomanip>
#include <windows.h>
#include "ntreg.h"
#include "sam.h"

// Full implementation of the Command Line Interface
// Handles argument parsing, auto-detection, and output formatting.

void printUsage(const std::string& progName) {
    std::cout << "NTPWEdit Clone CLI - Windows Password Management Tool\n";
    std::cout << "Usage: " << progName << " [options]\n";
    std::cout << "Options:\n";
    std::cout << "  -p, --path <path>    Specify path to SAM hive file\n";
    std::cout << "  -h, --help           Display this help message\n";
}

std::string autoDetectSAM() {
    WCHAR sysDir[MAX_PATH];
    if (GetSystemDirectory(sysDir, MAX_PATH)) {
        std::wstring ws(sysDir);
        ws += L"\\config\\SAM";
        char buf[MAX_PATH];
        WideCharToMultiByte(CP_ACP, 0, ws.c_str(), -1, buf, MAX_PATH, NULL, NULL);
        return std::string(buf);
    }
    return "";
}

int main(int argc, char* argv[]) {
    std::string samPath = "";

    for (int i = 1; i < argc; ++i) {
        std::string arg = argv[i];
```

```
        if (arg == "-h" || arg == "--help") {
            printUsage(argv[0]);
            return 0;
        } else if ((arg == "-p" || arg == "--path") && i + 1 < argc) {
            samPath = argv[++i];
        }
    }

    if (samPath.empty()) {
        std::cout << "[*] No path provided, attempting auto-detection...\n";
        samPath = autoDetectSAM();
    }

    if (samPath.empty()) {
        std::cerr << "[!] Error: Could not detect SAM path automatically.\n";
        return 1;
    }

    std::cout << "[*] Target SAM: " << samPath << "\n";

    Hive hive;
    if (!hive.open(samPath)) {
        std::cerr << "[!] Error: Failed to open SAM hive. (Try running as Administrator)\n";
        return 1;
    }

    SAMManager sam(&hive);
    if (!sam.loadUsers()) {
        std::cerr << "[!] Error: Failed to load users from SAM.\n";
        return 1;
    }

    const auto& users = sam.getUsers();
    std::cout << "\n" << std::left << std::setw(30) << "Username" << " | " << "RID" << "\n";
    std::cout << "----------------------------------------" << "\n";
    for (const auto& user : users) {
        std::cout << std::left << std::setw(30) << user.username << " | 0x" << std::hex <<
std::uppercase << user.rid << "\n";
    }

    std::cout << "\n[*] Successfully listed " << users.size() << " users.\n";
    return 0;
}
```

GLOSSARY OF TERMS

- **CLI (Command Line Interface)**: A text-based user interface used to interact with computer programs.
- **RID (Relative Identifier)**: The variable portion of a Security Identifier (SID) identifying a specific user or group.
- **SAM**: Security Accounts Manager.

- **Standard I/O**: The stream-based communication between a CLI tool and the host shell (stdout, stderr).
- **Static Linking**: Compiling all library code into the executable binary, eliminating external `.dll` dependencies.